WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

A PROJECT REPORT

on

KAIROS: AN AUTONOMOUS CAUSAL BUSINESS INTELLIGENCE AGENT FOR PROACTIVE MULTI-SOURCE METRIC MONITORING AND ROOT CAUSE ANALYSIS

Capstone Project-I

Bachelor of Technology (B.Tech.)

Computer Science and Engineering

Submitted by

Palla Chetana Reddy – 23WU0104050

Varshini Yadav Nasaram – 23WU0104067

Obulagari Nandini – 23WU0104049

Under the guidance of

Dr. K RajKumar

Assistant Professor,

Woxsen University.

School of Technology, Woxsen University, Hyderabad

October 2026

CERTIFICATE

This is to certify that the project report entitled **"KAIROS: An Autonomous Causal Business Intelligence Agent for Proactive Multi-Source Metric Monitoring and Root Cause Analysis"** submitted by Palla Chetana Reddy (23WU0104050), Varshini Yadav Nasaram (23WU0104067), Obulagari Nandini (23WU0104049), Bachelor of Technology, Woxsen University, is a bonafide record of work carried out by the students under my supervision and guidance in the course Capstone Project-I during the 7th semester of the academic year 2026–2027. The work embodied in this report has not been submitted elsewhere for a degree.

Signature of Mentor

Name: Dr. K RajKumar

Designation: Assistant Professor

Date: 07-10-2026

DECLARATION OF THE CANDIDATES

We hereby declare that the project work entitled **"KAIROS: An Autonomous Causal Business Intelligence Agent for Proactive Multi-Source Metric Monitoring and Root Cause Analysis"** submitted to the School of Technology, Woxsen University, for the degree of Bachelor of Technology is our original work, carried out under the guidance of Dr. K RajKumar. We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Signature of Student
Name: Palla Chetana Reddy
Roll Number: 23WU0104050

Signature of Student
Name: Varshini Yadav Nasaram
Roll Number: 23WU0104067

Signature of Student
Name: Obulagari Nandini
Roll Number: 23WU0104049

Date: 07-10-2026

ACKNOWLEDGMENT

We express our sincere gratitude to our project guide, Dr. K RajKumar, for the guidance, feedback and encouragement provided throughout this project, and for the questions raised during the reviews that shaped the most important parts of this work. We thank the evaluation panel of the second review, whose comments on data ingestion and large-file transfer led directly to the ingestion layer and the Fast Transfer service described in this report.

We thank the School of Technology, Woxsen University, for the computing resources, library access and the opportunity to carry out the Capstone Project. We acknowledge the open-source communities behind DuckDB, FastAPI, LangGraph, Prophet, DoWhy, causal-learn, pandas, React and Recharts, and the authors of the DYNOTEARS and LLM-Modulo papers, without whom this work would not have been possible. Classmates who tried early versions of the dashboard gave useful feedback on its design. Finally, we thank our families and friends for their patience and support.

Palla Chetana Reddy
Varshini Yadav Nasaram
Obulagari Nandini

ABSTRACT

Business intelligence (BI) tools such as Power BI and Tableau answer questions only after a person opens a dashboard and asks one. Problems can stay unnoticed for hours, and the explanations these tools offer are correlations rather than causes. This project presents **KAIROS**, an autonomous five-agent system that watches business metrics from many sources, detects anomalies, traces them to their causal origin and sends a verified plain-English alert without being asked.

Data enters through five lightweight protocol adapters (SQL, SFTP/FTP, files and object storage, REST and push), with aggregation pushed down to the source and a chunked, parallel, resumable transfer service for large files. Anomalies are scored with a robust seasonal z-score and an optional Prophet ensemble, DYNOTEARS learns a lagged causal graph of the metrics, and an LLM-Modulo verifier checks every number in the generated report before it is sent.

On a synthetic benchmark with known ground truth, causal discovery reached an F1 score of 0.97 against 0.90 for Granger regression and 0.50 for correlation, and the true incident origin was identified in all ten runs. A 106 MB raw export was transferred 6.4 times faster than single-stream FTP, or reduced to 193 KB by edge aggregation. The verifier caught 98.5% of corrupted numbers. The whole system runs on free, open-source software.

Keywords: proactive business intelligence, anomaly detection, causal discovery, DYNOTEARS, root cause analysis, multi-agent systems, LLM verification, data ingestion.

TABLE OF CONTENTS

CERTIFICATE 1
DECLARATION OF THE CANDIDATES 2
ACKNOWLEDGMENT 3
ABSTRACT 4
TABLE OF CONTENTS 5
LIST OF TABLES 6
LIST OF FIGURES 6
LIST OF ABBREVIATIONS 7
1. INTRODUCTION 8
  1.1 Background 8
  1.2 Problem Statement 8
  1.3 Significance 8
  1.4 Scope 9
  1.5 Organisation of the Report 9
2. TECHNOLOGY REVIEW 9
  2.1 Review Method 9
  2.2 Related Technologies 10
  2.3 Comparison and Analysis 11
  2.4 Review Summary 12
  2.5 Research Gap 12
3. OBJECTIVES AND METHODOLOGY 13
  3.1 Project Objectives 13
  3.2 Proposed Approach 13
  3.3 System Architecture 13
  3.4 Tools and Technologies 16
  3.5 Implementation Plan 17
  3.6 Novelty 17
4. IMPLEMENTATION AND RESULTS 18
  4.1 Implementation Details 18
  4.2 System Features 18
  4.3 Testing and Validation 19
  4.4 Results and Analysis 19
  4.5 Discussion 23
5. CONCLUSION AND FUTURE WORK 24
  5.1 Summary 24
  5.2 Objectives Achievement 24
  5.3 Limitations 24
  5.4 Lessons Learned 25
  5.5 Recommendations 25
  5.6 Future Scope for Capstone Project-II 25
REFERENCES 26
APPENDIX A: SOURCE CODE SNIPPETS 28
APPENDIX B: USER MANUAL 28
APPENDIX C: REPRODUCIBILITY AND BENCHMARK DETAILS 29

LIST OF TABLES

Table 1.1: Scope of Capstone Project-I 9
Table 2.1: Technology review: BI platforms and anomaly detection 10
Table 2.2: Technology review: causal analysis and agent orchestration 11
Table 2.3: Technology review: storage, ingestion and transfer 11
Table 2.4: Coverage of project requirements by existing approaches 12
Table 2.5: Research gaps identified from the technology review 12
Table 3.1: SMART objectives of the project 13
Table 3.2: Tools and technologies used 17
Table 4.1: Automated test suite 19
Table 4.2: Live ingestion demonstration 20
Table 4.3: Ingestion benchmark for a 105.6 MB raw export 20
Table 4.4: Anomaly detection over ten seeds (mean ± standard deviation) 21
Table 4.5: Causal discovery against ground truth over ten seeds 21
Table 4.6: Sensitivity of causal F1 and detection to their thresholds (mean of ten seeds) 22
Table 4.7: LLM-Modulo verifier stress test 23
Table 5.1: Achievement of the SMART objectives 24

LIST OF FIGURES

Figure 3.1: KAIROS system architecture 14
Figure 3.2: LangGraph state machine for one monitoring cycle 14
Figure 3.3: Ingestion routes by data location and size, with measured results 16
Figure 3.4: Fast Transfer protocol, including resume after a dropped connection 16
Figure 3.5: Project timeline (Gantt chart) for the 7th semester 17
Figure 4.1: Dashboard after a monitoring cycle on live demo sources 19
Figure 4.2: Fast Transfer and Reports pages 19
Figure 4.3: Transfer time and bytes on the wire for each method 20
Figure 4.4: Detected anomalies on four metrics, coloured by severity 21
Figure 4.5: Causal discovery results 22
Figure 4.6: Causal Analysis page with the CBMG and ranked root-cause chains 23

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
BI	Business Intelligence
CBMG	Causal Business Metric Graph
CDC	Change Data Capture
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DYNOTEARS	Dynamic NOTEARS (structure learning for time series)
ELT	Extract, Load, Transform
FTP / SFTP	File Transfer Protocol / SSH File Transfer Protocol
KPI	Key Performance Indicator
LLM	Large Language Model
MAD	Median Absolute Deviation
ProBI	Proactive Business Intelligence
RCA	Root Cause Analysis
REST	Representational State Transfer
SHD	Structural Hamming Distance
SLAC	Semantic Layer Auto-Construction
SMART	Specific, Measurable, Achievable, Relevant, Time-bound
SVAR	Structural Vector Autoregression
WAN	Wide Area Network

1. INTRODUCTION

Most organisations now hold their operating data in several places at once: an ERP database for orders, spreadsheets kept by the customer-support team, CSV exports from marketing tools, and REST APIs offered by logistics or payment providers. Business intelligence (BI) platforms such as Microsoft Power BI and Tableau [1] bring this data together into dashboards and have become standard equipment for managers. Their working model, however, has not changed in two decades: a person opens a dashboard, looks at a chart and decides whether something is wrong. If nobody looks, nothing is found.

1.1 Background

Two developments make a different model possible. Time-series anomaly detection has matured to the point where seasonal business metrics can be monitored automatically [2], [3], [4]. Causal discovery methods such as NOTEARS and its time-series extension DYNOTEARS learn directed, lagged relationships between variables from observational data [5], [6]. Large language models (LLMs) [7] can turn numerical findings into readable text, although they are known to state numbers that are not in their input [8]. Agent designs built on the ReAct pattern [9] and graph-based orchestration [10] make it practical to chain these steps into one autonomous loop.

1.2 Problem Statement

Current BI practice has three linked weaknesses. First, monitoring is reactive: an incident that starts on a Friday evening may only be seen on Monday. Second, the explanation features that do exist, such as Key Influencers in Power BI or Explain Data in Tableau, rank correlated fields and cannot say which metric moved first or which one caused the other. Third, connecting the data is expensive: managed ELT services charge by data volume, every company uses a different mix of systems, and moving large exports over FTP is slow.

During the second evaluation of this project, the review panel asked specifically how data from any source would be transferred, given that connectors are heavy and costly and FTP is slow for large files. The problem addressed is therefore to build a system that monitors multi-source business metrics without human initiation, identifies the causal origin of an anomaly, explains it accurately, and ingests data from heterogeneous systems at negligible cost.

1.3 Significance

Small and mid-sized businesses benefit most, because they rarely have a data team watching dashboards around the clock. Earlier detection shortens the time between a fault, such as a paused advertising campaign or a warehouse delay, and the corrective action. A causal explanation saves

analysts the manual work of tracing a chain of related metrics. Because KAIROS uses only free software and moves aggregated metrics instead of raw tables, its running cost stays close to zero.

1.4 Scope

The boundaries of the project are summarised in Table 1.1. Evaluation uses a synthetic benchmark with a known causal structure, because real company data with labelled root causes was not available within the semester.

Included	Excluded
Numeric business metrics as time series (hourly or finer)	Unstructured data such as text, images or documents
Ingestion from SQL, SFTP/FTP, files and object storage, REST and push clients	Streaming below one-minute latency
Anomaly detection, causal discovery and root-cause ranking	Multi-tenant deployment and billing
Verified report generation; Telegram, e-mail and dashboard delivery	User study with business staff
Edge aggregation and a resumable transfer service for large files	Deep anomaly models (Anomaly Transformer)

Table 1.1: Scope of Capstone Project-I

1.5 Organisation of the Report

Chapter 2 reviews existing technologies and identifies the research gap. Chapter 3 states the objectives and describes the methodology, architecture and novelty. Chapter 4 presents the implementation, testing and results. Chapter 5 concludes with limitations, lessons learned and the plan for Capstone Project-II.

2. TECHNOLOGY REVIEW

2.1 Review Method

Twelve technologies were reviewed across the functions KAIROS needs: analytics and anomaly detection, causal analysis and orchestration, and data storage and transfer. Each was assessed on the eleven criteria set by the Capstone guidelines, using vendor documentation, the original research papers and hands-on trials where the software was freely available. Similar technologies are compared side by side in Table 2.1, Table 2.2 and Table 2.3.

2.2 Related Technologies

Criterion	Power BI	Tableau	Prophet	Anomaly Transformer
Description	Microsoft BI and reporting platform [1].	Visual analytics platform owned by Salesforce.	Additive forecasting model from Meta [2].	Transformer for unsupervised time-series anomaly detection [11].
Key features	Power Query connectors, DAX, Key Influencers, anomaly markers per chart.	Drag-and-drop visuals, Explain Data, Pulse digests.	Trend, daily and weekly seasonality, uncertainty intervals.	Association discrepancy, minimax training.
Performance	In-memory engine; interactive on millions of rows.	Hyper engine; fast extracts.	Seconds per series on a laptop.	Strong results on public benchmarks.
Scalability	Premium capacity for large datasets.	Server and Cloud editions.	One model per series; parallelisable.	Needs GPU training per dataset.
Cost / licensing	Desktop free; Pro and Premium subscriptions.	Creator, Explorer and Viewer subscriptions.	Free, MIT licence.	Free research code.
Compatibility	Windows, web, mobile; hundreds of connectors.	Windows, macOS, web; many connectors.	Python and R; Stan backend.	Python, PyTorch.
Ease of use	Easy for charts; DAX is harder.	Moderate; strong community.	Easy; few parameters.	Hard; research-grade code.
Security	Azure AD, row-level security, encryption.	SSO, row-level security.	Library only.	Not applicable.
Current market	A market leader in enterprise BI.	Widely used in enterprises.	Popular for business forecasting.	Academic use.
Limitations	Reactive; explanations are correlational.	Explain Data lists factors, not causal chains.	Univariate; slower than robust statistics.	Long training windows; tuning per metric.
Relevance to project	Baseline of reactive BI; not used.	Baseline; not used.	Optional ensemble member in Agent 2.	Studied; deferred to Capstone-II.

Table 2.1: Technology review: BI platforms and anomaly detection

Criterion	CausalNex (DYNOTEARS)	Tigramite (PCMCI)	DoWhy	LangGraph
Description	QuantumBlack library for causal reasoning, including DYNOTEARS [6].	Tigramite package implementing PCMCI [12].	PyWhy library for causal effect estimation [13].	Graph-based orchestration for LLM agents [10].
Key features	Structure learning, Bayesian networks, lagged graphs.	Lagged conditional-independence tests, nonlinear tests.	Model, identify, estimate, refute workflow.	Typed state, conditional edges, retries.
Performance	Optimisation-based; fine for tens of variables.	Heavy with nonlinear tests.	Fast for linear estimators.	Negligible overhead.
Scalability	Cost grows cubically with variables.	Tens of variables.	Per-effect estimation.	Single process or server.
Cost / licensing	Free, Apache 2.0.	Free, GPL-3.0.	Free, MIT.	Free, MIT.
Compatibility	Pins older Python and pandas versions.	Python.	Python; pandas input.	Python and JavaScript.
Ease of use	Moderate.	Moderate; good documentation.	Clear API.	Moderate.
Security	Library only.	Library only.	Library only.	No built-in auth.
Current market	Niche, mainly consulting use.	Popular in climate science.	Widely cited in causal ML.	Fast-growing agent ecosystem.
Limitations	Dependency conflicts with current stack.	Copyleft licence; slow nonlinear tests.	Needs a graph as input; does not discover one.	API changes between major versions.

Criterion	CausalNex (DYNOTEARS)	Tigramite (PCMCI)	DoWhy	LangGraph
Relevance to project	Algorithm adopted; re-implemented in NumPy/SciPy.	Candidate for Capstone-II comparison.	Estimates effects on discovered edges.	Orchestrates the five agents.

Table 2.2: Technology review: causal analysis and agent orchestration

Criterion	DuckDB	Fivetran	Airbyte	IBM Aspera
Description	In-process analytical SQL database [14].	Managed ELT service [15].	Open-source ELT platform [16].	IBM high-speed transfer based on FASP [17].
Key features	Columnar, vectorised; reads CSV, Parquet, JSON and S3 directly.	Hundreds of maintained connectors, schema-drift handling.	Large connector catalogue, connector builder.	UDP transfer, rate control, resume, encryption.
Performance	Tens of millions of rows per second on one core.	Reliable scheduled syncs.	Depends on connector.	Near line rate on long links.
Scalability	Larger-than-memory processing.	Fully managed.	Kubernetes deployment.	Enterprise scale.
Cost / licensing	Free, MIT.	Priced per monthly active row.	Open-source core; paid cloud.	Proprietary, licensed.
Compatibility	Python, R, Java, Node; no server.	Cloud warehouses as destination.	Docker or Kubernetes required.	Dedicated clients and servers.
Ease of use	Easy; standard SQL.	Very easy.	Heavy to self-host.	Moderate.
Security	File-level; no user management.	SOC 2, encryption.	Self-managed.	Encryption in transit and at rest.
Current market	Rapidly growing adoption.	Market leader in managed ELT.	Large open-source community.	Media and life sciences.
Limitations	Single node; one writer process.	Cost grows with volume; copies whole tables.	Several containers per sync.	Licence cost; proprietary protocol.
Relevance to project	Metric store and file engine.	Rejected on cost.	Optional route for rare systems.	Design reference for Fast Transfer.

Table 2.3: Technology review: storage, ingestion and transfer

2.3 Comparison and Analysis

The commercial BI platforms are strong at presenting data but share one interaction model: they wait for a question. Their automated explanations are useful starting points, yet Key Influencers and Explain Data rank fields by statistical association, which cannot separate a cause from its effect or from a common driver [18]. Root-cause tools from the AIOps field, such as CloudRanger [19], MicroCause [20] and RCD [21], do use causal graphs, but they are built for microservice telemetry such as latency and error rates, and assume engineers as users. None of them produces business-facing explanations or reads ERP tables and spreadsheets.

Among causal discovery methods, Granger-style regression [22] is fast but tends to add edges between variables that share a driver. VARLiNGAM [23] learns lagged structure but relies on non-Gaussian noise, and PCMCI handles nonlinearity at a higher computational cost and under a copyleft licence. DYNOTEARS learns same-step and lagged edges together under an acyclicity constraint, which suits business metrics whose effects arrive with a delay. On the ingestion side, the choice is between paying per row, running a heavy self-hosted platform, or writing custom

code, while transfer tools such as Aspera solve the large-file problem but are proprietary. Table 2.4 summarises how existing approaches cover the requirements identified in Chapter 1.

Requirement	Dashboards (Power BI, Tableau)	AIOps RCA tools	ELT platforms	KAIROS
Monitoring without a human request	Partial (threshold alerts)	Yes	No	Yes
Directed, lagged causal explanation	No (correlational)	Yes (IT metrics)	No	Yes (business metrics)
Plain-English report with fact checking	No	No	No	Yes
Low-cost multi-source ingestion	Licensed connectors	No	Paid or heavy	Free and lightweight
Large-file transfer strategy	No	No	Bulk copy	Push-down, edge, Fast Transfer

Table 2.4: Coverage of project requirements by existing approaches

2.4 Review Summary

Three findings stand out. Monitoring, causal reasoning and narrative explanation exist separately in different tools, but no reviewed product combines them for business metrics. The open-source building blocks are mature and free: robust statistics and Prophet for detection, DYNOTEARS and DoWhy for causality, LangGraph for orchestration and DuckDB for storage. The cost of moving data depends mainly on how much is copied rather than on the protocol [24], so a design that moves aggregated metrics instead of raw rows can avoid both licence fees and long transfers.

2.5 Research Gap

The gaps found in the review are grouped by category in Table 2.5, with the evidence for each and the corresponding response in KAIROS.

Category	Gap in existing systems	Evidence	KAIROS response
Functional	BI tools do not monitor on their own initiative	Power BI and Tableau need a user session	Scheduled five-agent pipeline (ProBI)
Technical	Explanations are correlational, not causal or lagged	Key Influencers, Explain Data	DYNOTEARS causal graph (CBMG) with lag-aware ranking
Reliability	LLM narratives can state wrong numbers	Hallucination survey [8]	LLM-Modulo verifier on every report
Usability	Each new source needs manual schema mapping	ELT and semantic layers are hand-modelled	SLAC infers timestamp, shape and metrics
Cost	Connectors priced by volume or heavy to host	Fivetran pricing, Airbyte deployment	Five protocol adapters; aggregation at source
Performance	Single-stream FTP is slow for large files	TCP throughput limits [25]	Edge aggregation; parallel, resumable Fast Transfer

Table 2.5: Research gaps identified from the technology review

3. OBJECTIVES AND METHODOLOGY

3.1 Project Objectives

Five SMART objectives were set at the start of the semester and revised after the mid-term review, when the ingestion questions were raised. They are listed in Table 3.1.

No.	Objective	Measure of success	Deadline
O1	Build a five-agent pipeline that runs every 15 minutes without human initiation and stores data idempotently	Cycle under 30 s for 15 metrics; no duplicate rows on re-runs	Week 8
O2	Detect injected anomalies in seasonal business metrics	Event recall ≥ 95%; point precision ≥ 70%	Week 10
O3	Identify the causal origin of an incident	Causal F1 ≥ 0.85; origin correct in ≥ 90% of runs	Week 10
O4	Generate executive reports whose numbers are verified before delivery	≥ 95% of numeric misstatements caught	Week 11
O5	Ingest from at least four source types at low cost and handle large files	Incremental sync; a 100 MB export moved ≥ 3× faster than single-stream FTP	Week 11

Table 3.1: SMART objectives of the project

3.2 Proposed Approach

Development followed two-week Agile sprints with a working build at the end of each sprint. The first sprints produced the five agents and the dashboard shown at the mid-term review. After that review, the work concentrated on the ingestion layer, the causal engine and report verification. Quality was maintained through automated tests that run on every change, code review within the team, and benchmark scripts that regenerate every reported number from scratch.

3.3 System Architecture

The overall architecture is shown in Figure 3.1. Data from five kinds of sources passes through an ingestion layer into a DuckDB store. Five agents then run in sequence under a LangGraph state machine, triggered every 15 minutes by a background scheduler. A FastAPI [26] service exposes every function, and a React dashboard sits on top of it.

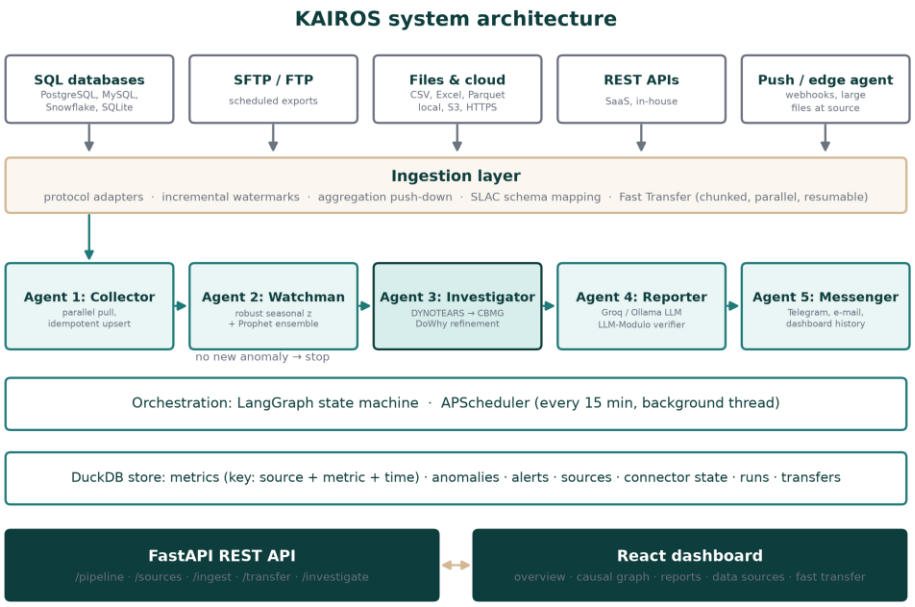


Figure 3.1: KAIROS system architecture

The control flow of one monitoring cycle appears in Figure 3.2. The graph stops after detection when no new alert-worthy anomaly is found, so a quiet cycle costs a fraction of a second and no alert is ever repeated.

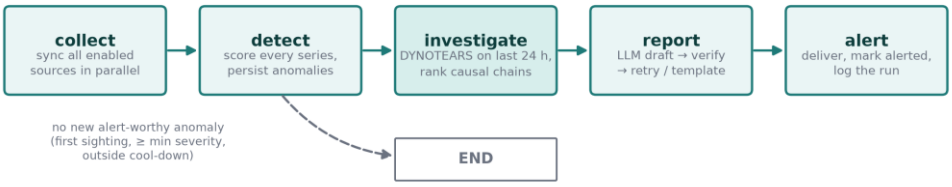


Figure 3.2: LangGraph state machine for one monitoring cycle

3.3.1 Agent 1: Data Collector

Every source implements one contract, `fetch(state)`, which returns rows in a canonical form (metric name, value, timestamp) together with an updated state. The state holds a watermark or the list of files already processed, so each cycle reads only new data. Rows are written with an upsert on the key (source, metric, timestamp), so a repeated sync updates values instead of duplicating them, a property that makes retries safe in any data pipeline [24]. Sources run in parallel with a timeout each, so one slow system cannot delay the others.

3.3.2 Agent 2: Anomaly Watchman

Each series is compared with an expected value made of a centred rolling median (the local level) plus an hour-of-day profile, which is kept only when it explains a meaningful share of the variation.

The residual is scaled by the median absolute deviation (MAD), giving a robust z-score $z = (x - m) / (1.4826 \times MAD)$ that is not distorted by the anomalies it is looking for [3]. Points with $|z| \geq 3.5$ are flagged; severity is low below 4.5, medium from 4.5, high from 6 and critical from 9 robust standard deviations. Prophet [2] can be enabled as a second detector. Every anomaly is stored, so it is reported as new exactly once.

3.3.3 Agent 3: Causal Investigator

The metrics are aligned on a common time grid, winsorised so that the anomalies do not distort the learned structure, and standardised. DYNOTEARS [6] then fits the structural vector autoregression $X_t = X_t W + X_{(t-1)} A + e$, where W holds same-step effects and A holds one-step lagged effects. It minimises the squared residual plus L1 penalties subject to the acyclicity constraint $h(W) = tr(e^{(W \circ W)}) - d = 0$ [5], using an augmented Lagrangian with L-BFGS-B. Edges with weight of at least 0.15 form the Causal Business Metric Graph (CBMG), and edge confidence is the share of bootstrap refits that keep each edge.

For each anomalous metric, the agent walks back through its ancestors in the CBMG and scores each chain by strength, confidence and whether the cause itself was anomalous shortly before the effect. An anomalous cause with no anomalous upstream cause is marked as the origin. DoWhy [13] then estimates the size of each direct effect.

3.3.4 Agent 4: NL Reporter

A prompt containing only computed facts is sent to an LLM, using the Groq API first and a local Ollama model as a fallback. Following the LLM-Modulo pattern [27], the draft is checked by a deterministic verifier: every number must match a computed fact at the precision written, and every metric name must exist. A failed draft is returned once with the list of errors; if it fails again, a template report that is correct by construction is used instead.

3.3.5 Agent 5: Messenger

The verified report is sent through the Telegram Bot API [28] and e-mail and stored in an alert history shown on the dashboard. A cooldown prevents repeated alerts for the same metric.

3.3.6 Ingestion Strategy for Any Source

The ingestion design answers the review question directly. Instead of one connector per product, KAIROS implements five protocol adapters that together cover almost any system. The guiding principle is to move the computation rather than the data. For a large table, the aggregation query runs inside the source database. For large raw files, a small edge agent aggregates them next to the data and pushes only KPIs. For small scheduled exports, SFTP/FTP [29] fetches only files that have not been seen before. Figure 3.3 shows how a source is routed. The Semantic Layer Auto-

Construction (SLAC) module inspects a sample of any new source and proposes its mapping (timestamp column, long or wide shape, metric columns, and whether raw records need aggregation), which removes most of the manual setup.

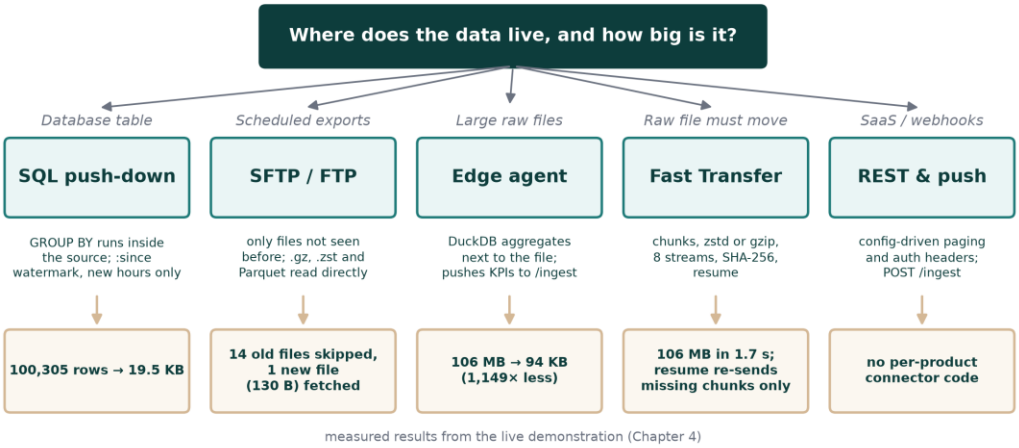


Figure 3.3: Ingestion routes by data location and size, with measured results

When a raw file must move, the Fast Transfer service is used (Figure 3.4). The file is split into chunks; each chunk is compressed with gzip or Zstandard [30], hashed, and sent in parallel with other chunks. A single TCP stream is limited by its window size and round-trip time [25], so parallel streams use more of the link [31]. If the connection drops, the client asks which chunks are missing and sends only those. The server verifies each chunk, writes it in place and re-hashes the whole file before accepting it.

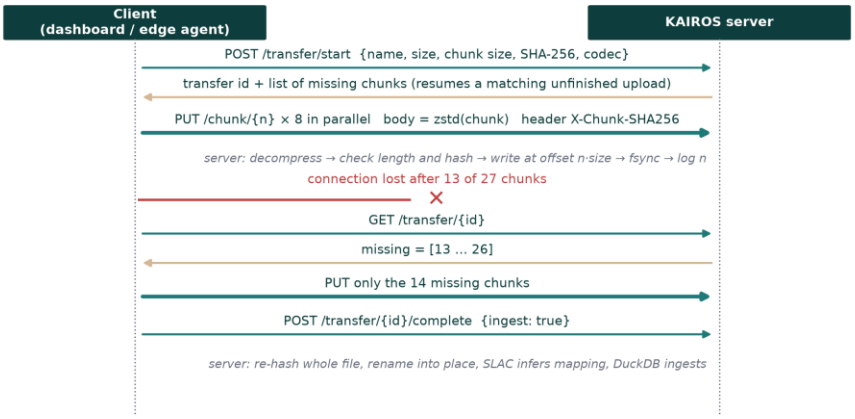


Figure 3.4: Fast Transfer protocol, including resume after a dropped connection

3.4 Tools and Technologies

The tools selected after the review are listed in Table 3.2. All of them are free and open source.

Layer	Tools
Languages	Python 3.13, JavaScript (ES2022), SQL
Backend and orchestration	FastAPI, Uvicorn, LangGraph, APScheduler, Pydantic
Data and storage	DuckDB, pandas, PyArrow, SQLAlchemy
Detection and causality	NumPy, SciPy (DYNOTEARS), Prophet, DoWhy, causal-learn (VARLiNGAM baseline), NetworkX
LLM and delivery	Groq API, Ollama (Llama 3.1 8B), Telegram Bot API, Gmail SMTP
Connectors and transfer	paramiko (SFTP), ftplib (FTP), httpx (REST), gspread (Sheets), zstandard, gzip
Frontend	React 18, Vite, Tailwind CSS, Recharts
Testing and tooling	pytest, pyftpdlib (test FTP server), Docker, Git, VS Code

Table 3.2: Tools and technologies used

3.5 Implementation Plan

The semester plan, with milestones at the three evaluations, is shown in Figure 3.5. Work on the agents and the dashboard ran in parallel among the three team members, and the ingestion and verification work was added after the second evaluation in response to the review.

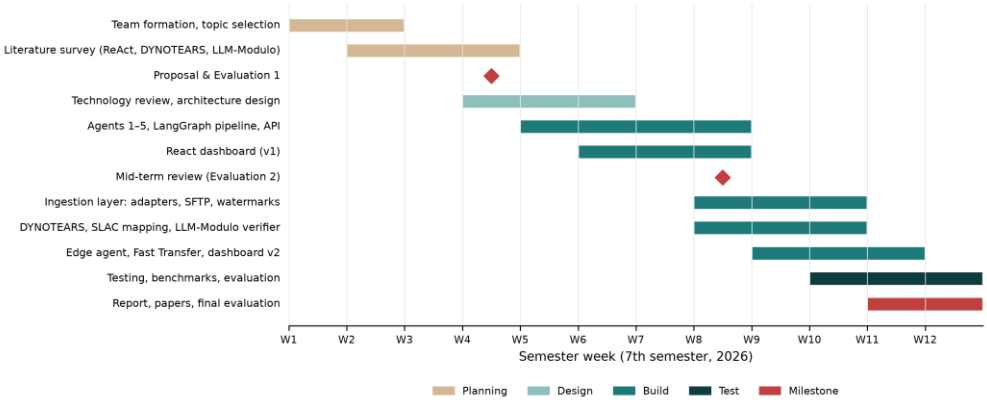


Figure 3.5: Project timeline (Gantt chart) for the 7th semester

3.6 Novelty

The project contributes four elements that were not found together in any reviewed system:

- **Proactive BI (ProBI):** a complete loop from ingestion to verified alert that runs on a schedule, stores every anomaly and alerts only on first sightings.
- **Causal Business Metric Graph (CBMG):** DYNOTEARS applied to business KPIs, with a ranking that combines edge strength, bootstrap confidence and the timing of anomalies to name the origin of an incident rather than a list of correlated metrics.
- **Verified narration:** an LLM-Modulo critic that checks numbers at the precision written, so a fluent but wrong report never reaches a manager.

- **Low-cost ingestion for any source:** protocol-level adapters, SLAC mapping, aggregation push-down, an edge agent and a resumable parallel transfer service, which together answer the cost and large-file concerns raised in the review.

The evidence for each of these claims is presented in Chapter 4.

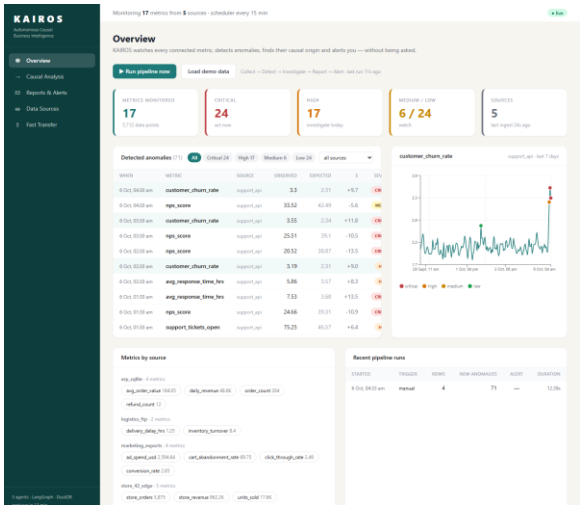
4. IMPLEMENTATION AND RESULTS

4.1 Implementation Details

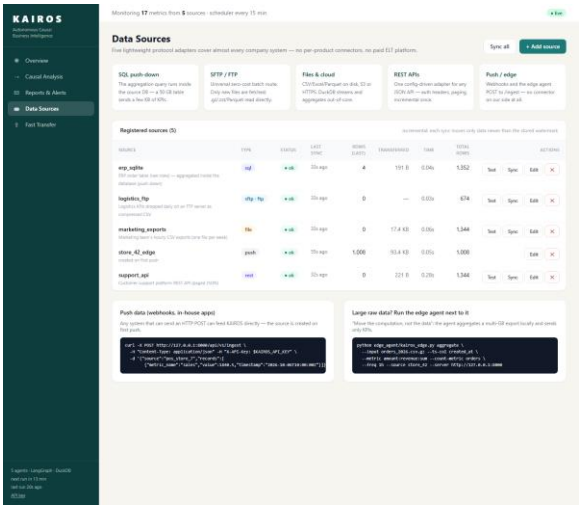
The backend consists of about 5,100 lines of Python and the dashboard of about 1,500 lines of JavaScript. Implementation proceeded in five steps: the DuckDB schema with primary keys and migrations; the connector framework and its adapters; the detection and causal models, including a NumPy/SciPy implementation of DYNOTEARS written because the existing library would not install on Python 3.13; the report verifier; and finally the transfer service, the edge agent and the dashboard. Every number in this chapter can be regenerated with two scripts in the repository, `benchmark/evaluate_models.py` and `benchmark/benchmark_ingestion.py`.

4.2 System Features

The dashboard has five pages, four of which are shown in Figure 4.1 and Figure 4.2. The Overview page shows severity counts, a filterable anomaly table and the time series of the selected metric with anomalies marked by severity. The Data Sources page lists every connected system with the rows and bytes moved in its last sync; new sources are added through a form whose Test button shows a sample and the mapping proposed by SLAC. The Fast Transfer page uploads files of any size from the browser with live chunk progress, pause and resume, and the Reports page shows the verified executive report and the alert history.

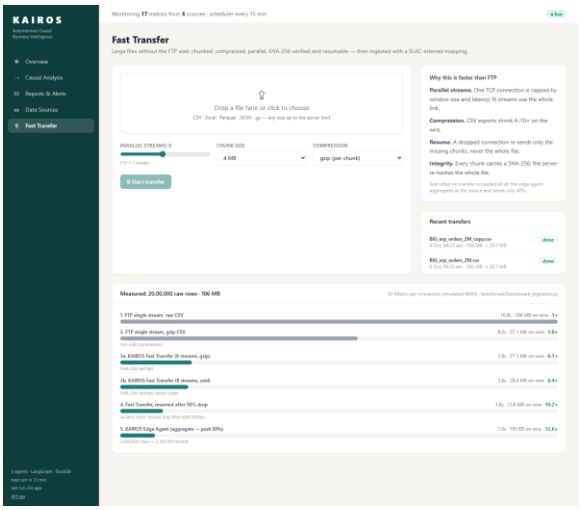


(a) Overview

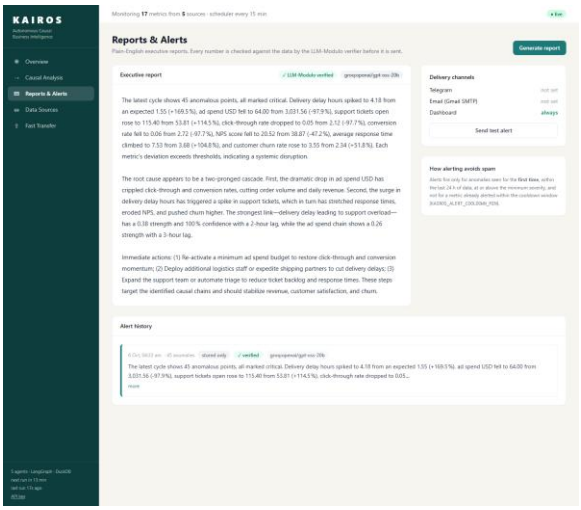


(b) Data Sources

Figure 4.1: Dashboard after a monitoring cycle on live demo sources



(a) Fast Transfer



(b) Reports & Alerts (LLM report marked as verified)

Figure 4.2: Fast Transfer and Reports pages

4.3 Testing and Validation

Testing combined automated tests, a live demonstration with simulated company systems, and two benchmark scripts. The automated suite is summarised in Table 4.1; all 36 tests pass. The connector tests start a real FTP server and a real HTTP server inside the test instead of using mocks.

Area	Tests	What is verified
Connectors	14	Long and wide mapping; CSV, gzip, Parquet and Excel; SQL push-down; REST paging; FTP new-file tracking; failure isolation
Core models	10	Upsert idempotency, detection, DYNOTEARS recovery, SLAC inference, verifier, alert gating
Fast Transfer	5	Chunk hashing, resume, corrupt-chunk rejection, whole-file checksum
API end to end	7	Seed, pipeline, no duplicate alerts, sources, push, upload, transfer, API key
Total	36	All passing

Table 4.1: Automated test suite

4.4 Results and Analysis

4.4.1 Multi-Source Ingestion

Four simulated company systems were connected: an ERP database with 100,305 raw order rows, a folder of weekly marketing CSV exports, an FTP server receiving daily compressed logistics files, and a paged REST API from a support platform. Table 4.2 records what happened during the live demonstration.

Step	Result
First sync of all four sources	4,704 KPI rows collected; the ERP sent 19.5 KB for 100,305 raw rows because the GROUP BY ran inside the database

Step	Result
Second sync	No new rows from FTP, files or REST; nothing duplicated
New file dropped on the FTP server	Only that file (130 bytes) downloaded; the 14 earlier files were skipped
Edge agent on a 2-million-row export (105.6 MB)	Aggregated locally in 0.6 s; 94.1 KB pushed, 1,149 times less data
Fast Transfer of the same file	Sent as 28.7 MB in 1.7 s; SHA-256 verified on the server
Connection cut after 13 of 27 chunks	Resume re-sent only the 14 missing chunks

Table 4.2: Live ingestion demonstration

4.4.2 Large-File Transfer Benchmark

To answer the concern about FTP, the same 2-million-row export (105.6 MB) was moved by each method between two processes on one machine, with every connection capped at 50 Mbit/s to emulate a wide-area link. The cap was applied in the same way to FTP and to Fast Transfer. The results are given in Table 4.3 and Figure 4.3.

Method	Time (s)	Bytes sent	Speed-up
FTP, single stream, raw CSV	16.90	105.6 MB	1.0×
FTP, single stream, gzip (incl. 4.8 s compression)	9.18	27.1 MB	1.8×
Fast Transfer, 8 streams, gzip	2.76	27.1 MB	6.1×
Fast Transfer, 8 streams, zstd	2.63	29.4 MB	6.4×
Fast Transfer, resumed after a 50% drop	1.65	13.6 MB	10.2×
Edge agent (aggregate, push KPIs)	1.34	192.7 KB	12.6×

Table 4.3: Ingestion benchmark for a 105.6 MB raw export

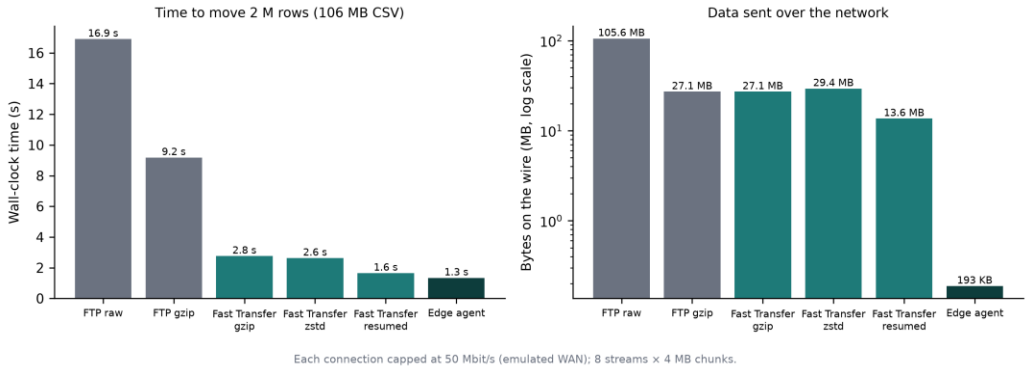


Figure 4.3: Transfer time and bytes on the wire for each method

Fast Transfer was 6.4 times faster than plain FTP while sending about a quarter of the bytes, and an interrupted transfer cost only the missing half. Edge aggregation was faster still and reduced network traffic by a factor of about 550. This supports the central argument that most large-data cases should not transfer raw data at all.

4.4.3 Anomaly Detection

Detection was evaluated over ten random seeds of the synthetic benchmark (15 metrics from four sources, 336 hourly points, two cascading incidents and six isolated anomalies). An event counts as detected if any point of the affected metric is flagged inside the event window. Table 4.4 gives the results and Figure 4.4 shows typical output.

Detector	Event recall	Point precision	Time per cycle (s)
Robust seasonal z-score	100.0 ± 0.0%	77.2 ± 6.2%	0.23
Robust z-score + Prophet ensemble (5 seeds)	100.0 ± 0.0%	75.2 ± 4.1%	3.34

Table 4.4: Anomaly detection over ten seeds (mean ± standard deviation)

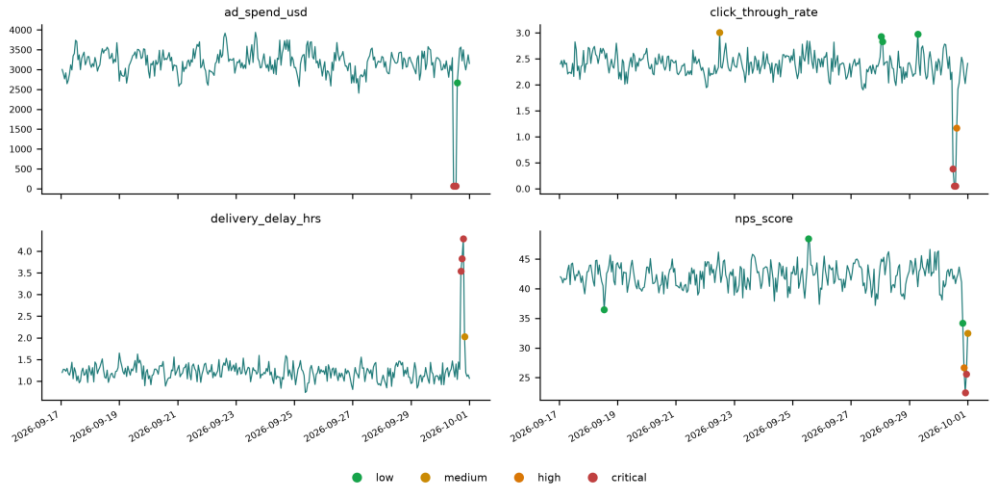


Figure 4.4: Detected anomalies on four metrics, coloured by severity

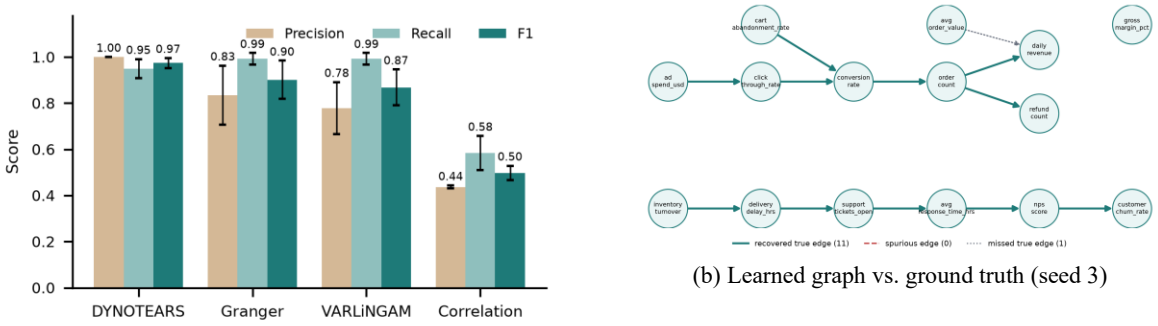
Every injected event was found. The flags counted as false positives are mostly points just outside the event windows, where the cascade is still settling. Prophet did not improve precision on this benchmark and was about fourteen times slower, so it is kept as an option rather than the default.

4.4.4 Causal Discovery and Root Cause

The learned graphs were compared with the twelve true causal edges of the benchmark. Table 4.5 and Figure 4.5 compare DYNOTEARS with three baselines.

Method	Precision	Recall	F1	SHD
DYNOTEARS (KAIROS)	1.00 ± 0.00	0.95 ± 0.04	0.97 ± 0.02	0.6 ± 0.5
Granger (lagged OLS)	0.83 ± 0.13	0.99 ± 0.03	0.90 ± 0.08	2.8 ± 2.5
VARLiNGAM	0.78 ± 0.11	0.99 ± 0.03	0.87 ± 0.08	3.7 ± 2.5
Pearson correlation	0.44 ± 0.01	0.58 ± 0.07	0.50 ± 0.03	7.0 ± 0.9

Table 4.5: Causal discovery against ground truth over ten seeds



(a) Precision, recall and F1 (ten seeds)

(b) Learned graph vs. ground truth (seed 3)

Figure 4.5: Causal discovery results

DYNOTEARS never added a false edge in ten runs; its occasional miss was the weakest true link, from average order value to revenue. The baselines found almost every true edge but added spurious ones, and correlation, which has no direction, performed worst. Figure 4.5(b) shows a learned graph against the ground truth. In the full pipeline, KAIROS named both injected origins, advertising spend and delivery delay, in all ten runs, and an investigation took 1.6 s on average.

The sensitivity of these results to the two main settings is shown in Table 4.6. The chosen values ($\lambda = 0.05$ for the L1 penalty, $\tau = 0.15$ for the edge threshold, $z = 3.5$ for detection) sit in a stable region; raising the detection threshold to 4.0 trades a little recall for much higher precision.

Setting	Value	Result
DYNOTEARS, $\lambda = 0.05$	τ = 0.10 / 0.15 / 0.20 / 0.30	F1 = 0.928 / 0.974 / 0.942 / 0.899
DYNOTEARS, $\tau = 0.15$	λ = 0.01 / 0.05 / 0.10	F1 = 0.845 / 0.974 / 0.942
Detection threshold z	3.0 / 3.5 / 4.0 / 5.0	Precision = 0.54 / 0.77 / 0.93 / 1.00
Detection threshold z	3.0 / 3.5 / 4.0 / 5.0	Event recall = 1.00 / 1.00 / 0.99 / 0.99

Table 4.6: Sensitivity of causal F1 and detection to their thresholds (mean of ten seeds)

The same analysis as it appears on the dashboard is shown in Figure 4.6. Each chain lists its strength, bootstrap confidence, lag and whether the cause moved before the effect.

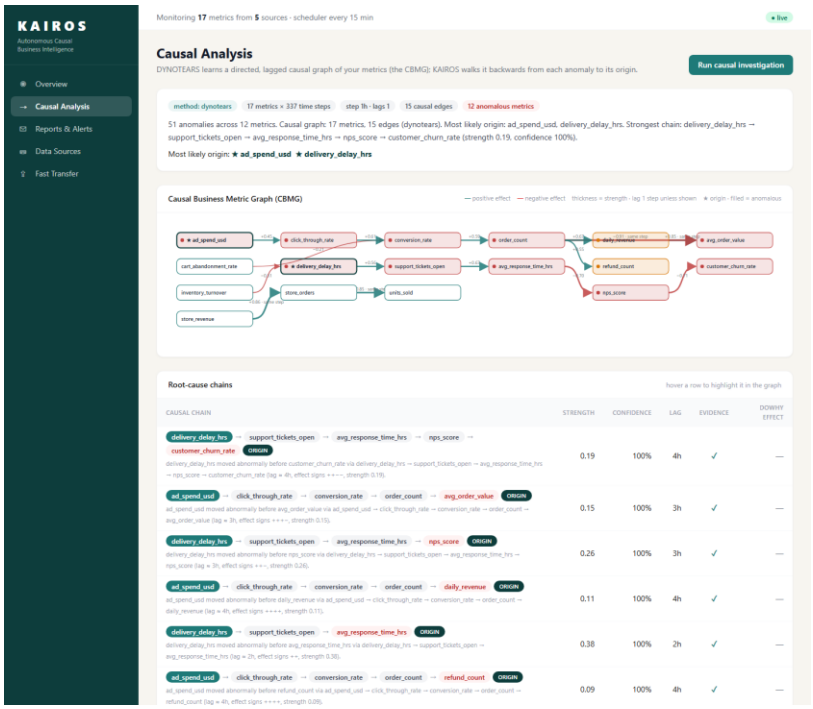


Figure 4.6: Causal Analysis page with the CBMG and ranked root-cause chains

4.4.5 Report Verification

The verifier was tested by corrupting numbers and metric names in correct reports; the results are in Table 4.7. An earlier version accepted 7.03 as a match for 6.71 because both round to 7. Comparing at the precision actually written raised the catch rate from 61% to 98.5% without rejecting faithful rounding.

Test	Cases	Result
Faithful reports accepted	5	100%
Corrupted numbers caught	200	98.5%
Invented metric names caught	50	100%
Faithful one-decimal rounding accepted	200	100%

Table 4.7: LLM-Modulo verifier stress test

In the live run, the report written by the Groq model passed verification on its first attempt (Figure 4.2(b)). The full five-agent cycle on live sources took 12.4 s.

4.5 Discussion

The results support the two main claims of the project. First, causal discovery gives better explanations than the correlation logic behind current BI features: on data with a known structure it identified the origin of every incident, which is the work an analyst would otherwise do by hand. Second, the cost and speed of ingestion depend mostly on what is moved. Aggregating at the source

turned the large-file problem raised in the review into a transfer of a few hundred kilobytes, and Fast Transfer covers the cases where the raw file is genuinely needed.

Two cautions apply. The benchmark data follows a linear model with known lags, which favours DYNOTEARS, and the transfer link was emulated on one machine. Both points are listed as limitations in Chapter 5.

5. CONCLUSION AND FUTURE WORK

5.1 Summary

KAIROS shows that business monitoring can move from a pull model, in which someone has to ask, to a push model in which the system finds the problem, explains its cause and reports it in verified language. The project delivered a working five-agent system, a web dashboard, an edge agent, a transfer service, 36 automated tests and two reproducible benchmarks.

5.2 Objectives Achievement

All five objectives were met, as summarised in Table 5.1.

Objective	Target	Achieved
O1 Autonomous pipeline	< 30 s per cycle, no duplicates	12.4 s; idempotent upsert verified by tests
O2 Anomaly detection	≥ 95% recall, ≥ 70% precision	100% recall, 77.2% precision
O3 Causal origin	F1 ≥ 0.85, origin ≥ 90%	F1 0.97, origin 100%
O4 Verified reports	≥ 95% misstatements caught	98.5% caught, 100% faithful accepted
O5 Low-cost ingestion	≥ 4 source types, ≥ 3× faster than FTP	5 adapter types; 6.4× (edge 12.6×)

Table 5.1: Achievement of the SMART objectives

5.3 Limitations

- Evaluation used synthetic data whose linear, lagged structure matches the assumptions of DYNOTEARS; results on real company data may be lower.
- DYNOTEARS assumes no hidden common causes and linear effects, so a driver that KAIROS does not monitor can appear as a spurious link.
- The transfer benchmark ran on one machine with an emulated 50 Mbit/s link; tests over real long-distance links remain to be done.
- The verifier checks numbers and metric names but not qualitative claims; in one live report the LLM described all anomalies as critical, which was not fully correct and was not caught.

- The Anomaly Transformer was reviewed but not implemented, and no user study with business staff has been carried out.

5.4 Lessons Learned

The most useful lesson came from the review itself: a question about FTP led to the realisation that the amount of data moved matters more than the protocol used to move it. Writing DYNOTEARS directly in NumPy and SciPy took longer than installing a library, but it removed a dependency conflict and made the method easier to explain. Testing against a real FTP server instead of mocks caught two bugs in file tracking, and keeping every number reproducible through scripts made the results easier to defend. Within the team, dividing the work by agent and integrating at sprint boundaries worked well, although integration should have started earlier.

5.5 Recommendations

Before production use, KAIROS should run in shadow mode on one real business unit for several weeks, with analysts marking each alert as useful or not. SFTP or FTPS should always be used instead of plain FTP, with read-only accounts.

5.6 Future Scope for Capstone Project-II

- Evaluate on public datasets (Olist e-commerce, Superstore) and on data from a partner company.
- Add nonlinear causal discovery (PCMCI) and test robustness to hidden confounders.
- Implement the Anomaly Transformer and compare it with the current detector.
- Extend the verifier to qualitative claims such as severity and direction of change.
- Measure Fast Transfer over real long-distance links, and add change data capture [32] for databases.
- Add multi-tenant deployment, role-based access and a user study with managers.

In the longer term, KAIROS could become a low-cost monitoring service for small businesses that cannot afford a data team, giving them the kind of root-cause analysis that today requires an analyst.

REFERENCES

[1] "Power BI documentation: Key influencers visual," Microsoft. [Online]. Available: https://learn.microsoft.com/power-bi/visuals/power-bi-visualization-influencers. [Accessed: Oct. 6, 2026].

[2] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[3] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *Journal of Experimental Social Psychology*, vol. 49, no. 4, pp. 764–766, 2013.

[4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[5] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, "DAGs with NO TEARS: Continuous optimization for structure learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[6] R. Pamfil, N. Sriwattanaworachai, S. Desai, P. Pilgerstorfer, K. Georgatzis, P. Beaumont, and B. Aragam, "DYNOTEARS: Structure learning from time-series data," in *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR vol. 108, 2020, pp. 1595–1605.

[7] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[8] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, Kigali, Rwanda, 2023.

[10] "LangGraph documentation," LangChain, Inc. [Online]. Available: https://langchain-ai.github.io/langgraph/. [Accessed: Oct. 6, 2026].

[11] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly Transformer: Time series anomaly detection with association discrepancy," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[12] J. Runge, P. Nowack, M. Kretschmer, S. Flaxman, and D. Sejdinovic, "Detecting and quantifying causal associations in large nonlinear time series datasets," *Science Advances*, vol. 5, no. 11, eaau4996, 2019.

[13] A. Sharma and E. Kiciman, "DoWhy: An end-to-end library for causal inference," arXiv:2011.04216, 2020.

[14] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Amsterdam, Netherlands, 2019, pp. 1981–1984.

[15] "Fivetran pricing: Monthly active rows," Fivetran. [Online]. Available: https://fivetran.com/docs/usage-based-pricing. [Accessed: Oct. 6, 2026].

[16] "Airbyte documentation," Airbyte, Inc. [Online]. Available: https://docs.airbyte.com/. [Accessed: Oct. 6, 2026].

[17] "IBM Aspera FASP high-speed transport," IBM. [Online]. Available: https://www.ibm.com/products/aspera. [Accessed: Oct. 6, 2026].

[18] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[19] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "CloudRanger: Root cause identification for cloud native systems," in *Proc. 18th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGRID)*, 2018, pp. 492–502.

[20] Y. Meng, S. Zhang, Y. Sun, R. Zhang, Z. Hu, Y. Zhang, C. Jia, Z. Wang, and D. Pei, "Localizing failure root causes in a microservice through causality inference," in *Proc. IEEE/ACM 28th Int. Symp. Quality of Service (IWQoS)*, 2020.

[21] M. A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, "Root cause analysis of failures in microservices through causal discovery," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[22] C. W. J. Granger, "Investigating causal relations by econometric models and cross-spectral methods," *Econometrica*, vol. 37, no. 3, pp. 424–438, 1969.

[23] A. Hyvärinen, K. Zhang, S. Shimizu, and P. O. Hoyer, "Estimation of a structural vector autoregression model using non-Gaussianity," *Journal of Machine Learning Research*, vol. 11, pp. 1709–1731, 2010.

[24] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[25] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM Computer Communication Review*, vol. 27, no. 3, pp. 67–82, 1997.

[26] "FastAPI documentation," S. Ramírez. [Online]. Available: https://fastapi.tiangolo.com/. [Accessed: Oct. 6, 2026].

[27] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, and A. Murthy, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR vol. 235, 2024.

[28] "Telegram Bot API," Telegram. [Online]. Available: https://core.telegram.org/bots/api. [Accessed: Oct. 6, 2026].

[29] J. Postel and J. Reynolds, "File Transfer Protocol (FTP)," IETF RFC 959, Oct. 1985.

[30] Y. Collet and M. Kucherawy, "Zstandard compression and the application/zstd media type," IETF RFC 8878, Feb. 2021.

[31] T. J. Hacker, B. D. Athey, and B. Noble, "The end-to-end performance effects of parallel TCP sockets on a lossy wide-area network," in *Proc. 16th Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2002.

[32] "Debezium documentation," Red Hat. [Online]. Available: https://debezium.io/documentation/. [Accessed: Oct. 6, 2026].

APPENDIX A: SOURCE CODE SNIPPETS

A.1 DYNOTEARS objective and gradient (`backend/models/dynotears.py`). The weights are split into positive and negative parts so that the L1 penalty becomes linear and L-BFGS-B can be used with simple bounds.

```
def objective(x, rho, alpha):
    W, A = unpack(x)
    R    = Xt - Xt @ W - Xlags @ A                # SVAR residual
    loss = 0.5 / n * np.sum(R ** 2)
    h, E = h_func(W)                              # h(W) = tr(exp(W*W)) - d
    obj  = loss + 0.5 * rho * h * h + alpha * h + l1_penalty(x)
    g_w  = -1.0 / n * Xt.T @ R + (rho * h + alpha) * (E.T * 2 * W)
    g_a  = -1.0 / n * Xlags.T @ R
    return obj, pack_gradient(g_w, g_a)
```

A.2 Verifying a number at the precision it was written (`backend/agents/nl_reporter.py`).

```
def _close(value: float, allowed: float, decimals: int = 2) -> bool:
    # 3.97 may be written 3.97, 4.0 or 4, but never 3.78 or 7.03
    step = 0.5 * 10 ** (-decimals)
    return abs(value - allowed) <= step + abs(allowed) * 1e-3 + 1e-9
```

A.3 Idempotent, incremental storage (`backend/db.py`).

```
INSERT INTO metrics (source, metric_name, timestamp, value)
SELECT source, metric_name, timestamp, value FROM _kairos_batch
ON CONFLICT (source, metric_name, timestamp)
DO UPDATE SET value = excluded.value, ingested_at = now()
```

APPENDIX B: USER MANUAL

B.1 Installation

```
git clone <repository> kairos && cd kairos
python -m venv backend/venv && backend\venv\Scripts\activate
pip install -r backend/requirements.txt
cd frontend && npm install && npm run build && cd ..
copy .env.example .env      # add GROQ_API_KEY, TELEGRAM_BOT_TOKEN (optional)
uvicorn backend.main:app     # dashboard and API on http://127.0.0.1:8000
```

B.2 Using the dashboard

1. Open **Data Sources** and click **Add source**. Choose a type (SQL, file, SFTP/FTP, REST or push), fill in the connection details and click **Test & preview**. Apply the mapping suggested by SLAC and save.
2. Click **Sync all** to collect data. The table shows the rows and bytes moved for each source.
3. On **Overview**, click **Run pipeline now** or wait for the 15-minute schedule. Click any anomaly to see its chart.

4. Open **Causal Analysis** to see the CBMG and the ranked root-cause chains; hover a chain to highlight it.
5. Open **Reports & Alerts** to read the verified report and the alert history.
6. For large files, use **Fast Transfer**: drop the file, choose streams and compression, start, and click **Ingest** when it completes.

B.3 Edge agent (run next to large data inside the company network)

```
python edge_agent/kairos_edge.py aggregate --input orders.csv.gz --ts-col created_at \
    --metric amount:revenue:sum --count-metric orders --freq 1h \
    --source store_42 --server https://kairos.example.com
python edge_agent/kairos_edge.py transfer --file dump.parquet --workers 8 --compress zstd --ingest
```

APPENDIX C: REPRODUCIBILITY AND BENCHMARK DETAILS

C.1 Reproducing the results. Every figure and table in Chapter 4 is produced by the commands below. Random seeds are fixed, so the numbers repeat exactly on the same machine; timings vary with hardware.

```
python benchmark/evaluate_models.py --seeds 10
python benchmark/benchmark_ingestion.py --rows 2000000 --stream-mbps 50
pytest tests -q                          # 36 passed
python docs/report/make_figures.py       # all figures
```

C.2 Ground-truth causal structure of the benchmark. Marketing chain: advertising spend → click-through rate → conversion rate → order count → daily revenue, with cart abandonment (negative) into conversion, average order value into revenue, and order count into refunds. Operations chain: inventory turnover (negative) → delivery delay → open support tickets → response time → NPS (negative) → churn (negative). Gross margin is independent. Two incidents were injected in the last 24 hours (a three-hour collapse in advertising spend and a three-hour spike in delivery delay), plus six isolated anomalies of graded severity.

C.3 Run times. On a laptop (Python 3.13, Windows 11), detection for 15 series took 0.23 s, DYNOTEARS 0.25 s, a full investigation with two bootstrap refits and DoWhy 1.6 s, and a complete five-agent cycle including an LLM call 12.4 s.